Assignment 3:

Object-Oriented Software Engineering

Short Answer Questions

1. Define the term 'Encapsulation' and state its primary benefit in software engineering.
2. What is the main difference between a Class diagram and an Object diagram in UML?
3. Define 'Functional Requirements' and provide one clear example.
4. What is the purpose of an Abstract Class in object-oriented design?
5. State the core objective of a Use Case Description.
6. What is the difference between 'Aggregation' and 'Composition' relationships?
7. Define 'Cohesion' and state why high cohesion is desired in software modules.
8. What role does a 'Boundary Object' play in robustness analysis?
9. Explain the basic goal of Polymorphism.
10. What is the difference between Unit Testing and Integration Testing in an object-oriented context?
11. Explain the fundamental difference between a 'Program' and a 'Software Product' as defined in Unit 1.
12. What does the 'Open-Closed Principle' (OCP) stand for in object-oriented design?
13. Differentiate between 'Functional Requirements' and 'Non-Functional Requirements' by framing a one-liner example of each.
14. State Karner's core rule for selecting the historical baseline of 20 person-hours per Use Case Point (UCP).
15. What are the primary metrics used to compute a single class's complexity weight using the Class Point estimation method?
16. Define 'Record Element Types' (RET) and 'Data Element Types' (DET) when mapping OO classes onto Internal Logical Files (ILFs) under the OOFP method.
17. Write the mathematical formula to calculate total Risk Exposure (RE) during risk projection.
18. What is the structural layout distinction between a Sequence Diagram and a Collaboration Diagram when modeling the same dynamic interaction?
19. Explain the purpose of using a 'Fork Bar' and a 'Join Bar' inside a UML Activity Diagram.
20. State the core formula used to calculate a system's 'Availability' percentage based on its uptime and downtime history.

Detailed Answer Questions

1. Compare the traditional Waterall process model with the Object-Oriented Software Development lifecycle. Detail how iterative development mitigates risks early.
2. Case Study: A university requires an automated Online Course Registration System. Students must be able to view course catalogs, register for courses, and drop courses online. Professors must be able to view their student rosters and input final grades. The system must also interface with the university's billing system.
    a. Identify all actors and write out the comprehensive Use Case Diagram for this scenario.
3. Explain the purpose and components of a UML Sequence Diagram. Draw a sequence diagram illustrating the process of an ATM cash withdrawal request.
4. Describe the concept of 'Coupling'. Discuss three distinct types of coupling found in object-oriented systems and explain strategies to minimize them.
5. Detail the steps involved in transforming an Analysis Model into a System Design Model. Explain how subsystem decomposition is achieved.
6. Discuss the distinct challenges encountered when testing object-oriented software compared to procedural software. Address how inheritance and polymorphism affect test coverage.
7. Elaborate on the 'Layers of Software Engineering'. Draw a neat diagram showing how the layers stack and interact (Quality Focus, Process, Methods, Tools).
8. Case Study (Estimation): You are tasked with sizing a new Hospital Appointment Management System. The requirements reveal 24 core Domain classes, a standard form-based GUI interface, and an empirical productivity setting of 16 person-days per class. Using the Lorenz and Kidd Estimation Method, compute the support classes, total classes, and final effort in person-months (assume a standard 20 person-days per month convention). Detail every step clearly.
9. Case Study (Use Case Points): A team designs an AI-Assisted Recruitment Tool. It utilizes 2 simple actors (external APIs) and 2 complex actors (human users via a web application). The use-case layout features 8 simple use cases ($\leq$3 transactions), 4 average use cases (4–7 transactions), and 2 complex use cases (>7 transactions). If the Technical Complexity Factor (TCF) is calculated at 0.95 and the Environmental Factor (EF) is 0.85, compute the Total Unadjusted Use Case Points and final development effort in person-hours using Karner's standard productivity rate.
10. Analyze the Class Point Method lifecycle workflow. Present the classification system of the four structural component types (PDT, HIT, DMT, TMT) and layout the complete mathematical step-by-step procedure used to adjust Total Unadjusted Class Points (TUCP) into final Adjusted Class Points (CFP).
11. Compare White-Box Reuse against Black-Box Reuse inside an object-oriented design space. Detail how Inheritance and Composition serve as distinct reuse mechanisms. Support your comparative layout with structural UML schemas for each, highlighting why architectural trends prefer composition over inheritance.
12. Explain the structural design of UML Class Diagrams. Differentiate explicitly between Association, Aggregation, and Composition relationships. Illustrate each

connection format using clear visual blocks with structural line notations and diamond/triangle decorators as prescribed by UML standards.

13. Detail the elements, tasks, and effectiveness metrics of a formal Software Quality Assurance (SQA) function. Discuss why organizational independence is critical for an SQA group to successfully balance timeline pressures.
14. Case Study (Risk Management): Create a risk assessment framework for a system with late-stage requirements volatility. Draft an RMMM (Mitigation, Monitoring, and Management) Plan for the risk "Requirements change late in the iteration". Detail specific, measurable indicators to watch and outline a robust contingency path should the risk materialize.